

W3.io Protocol

Verifiable Workflow Execution with On-Chain Settlement

Audie Sheridan

Ben Murray

David Post

2026

Contents

1	Abstract	1
2	Introduction	2
2.1	The Integration Gap	2
2.2	W3.io's Approach	3
2.3	What This Paper Covers	4
3	Architecture	5
3.1	Ingredients, Recipes, and Solutions	5
3.2	Actions and Workflows	5
3.3	The Three-Layer Data Architecture	7
3.4	The Spine and Ribs	7
4	Workflows	8
4.1	Declarative Syntax	8
4.2	Triggers	10
4.3	Step Kinds	11
4.4	Expressions and Data Flow	12
4.5	Conditionals and Failure Handling	13
4.6	Compilation and Deployment	13

1 Abstract

W3.io is a workflow execution protocol that connects fragmented Web3 infrastructure into verifiable, multi-step workflows that power real-world applications. Developers compose workflows from

modular actions — blockchain operations across Ethereum, Solana, and Bitcoin, data queries, AI inference, payments, compliance checks, and dozens of other ecosystem partner capabilities — using a declarative YAML syntax compatible with GitHub Actions. The protocol transforms isolated infrastructure components into programmable, composable pipelines where each step’s execution is independently verifiable.

A distributed network of validators executes each workflow step, reaches Byzantine fault-tolerant consensus on the result, and produces cryptographic settlement receipts. These receipts are accumulated into epoch commitments anchored to an EVM L1, enabling any party to verify that a specific workflow executed correctly without re-executing it or trusting the executor.

The W3 token gates participation in the network. Ecosystem partners post collateral to operate subnets, solution builders stake to deploy applications, sales teams stake to earn rewards for driving adoption and end-user conversions, and validators bond tokens to secure consensus. Revenue from workflow execution flows in stablecoins, separating the utility token from the payment medium. Rewards are tied to active contribution — execution, verification, building, and adoption — not passive holding. The token does not represent equity, debt, or a claim on protocol revenues.

This paper describes the architecture, workflow execution model, consensus mechanism, settlement layer, namespace model, validator infrastructure, token utility, and security properties of the W3.io protocol. The protocol is implemented in Rust, open-source, and deployed on testnet with over 200,000 workflow executions per day.

2 Introduction

2.1 The Integration Gap

Decentralized infrastructure has matured rapidly. Individual services now exist for payments, data, compute, storage, identity, oracles, and security. Multiple blockchains provide settlement for different use cases. The building blocks are there.

What doesn’t exist is a way to connect these services into verifiable end-to-end workflows. A business process that requires a payment, a compliance check, a data query, and an on-chain settlement today requires custom integration code for each service, separate authentication and billing relationships with each provider, and no unified way to verify that the entire process executed correctly.

This gap has three dimensions.

Centralization by default. When developers need to orchestrate multiple services, the path of least resistance is a centralized server calling each API sequentially. This contradicts the decentralized properties of the underlying services, introduces single points of failure, and creates vendor

lock-in with cloud providers. The result is decentralized infrastructure coordinated by centralized glue code.

Integration complexity. Each decentralized service has its own API patterns, authentication mechanisms, data formats, and error handling conventions. Integrating even two services requires specialized knowledge of both. Integrating five or ten, as real business processes demand, multiplies the development cost, the testing surface, and the ongoing maintenance burden. Most Web3 development is bespoke, built from scratch against the same patterns that every other team is also building from scratch.

Business friction. Beyond the technical challenges, using multiple decentralized providers means separate legal agreements, separate payment arrangements, and separate vendor relationships. Discovery is also a problem. Hundreds of decentralized projects exist, but their capabilities and integration surfaces are poorly documented and difficult to evaluate without deep technical engagement.

These challenges are not hypothetical. They are the reason most production Web3 applications still run their off-chain logic on AWS, even when the on-chain components are fully decentralized.

2.2 W3.io's Approach

W3.io addresses the integration gap with a protocol-level execution layer that connects decentralized services into verifiable workflows. Rather than requiring developers to write custom integration code, W3.io provides a declarative workflow language where each step invokes a pre-integrated action from the W3.io ecosystem.

W3.io's model has three layers.

Actions are modular capabilities provided by ecosystem partners. An action encapsulates a specific operation behind a standard interface: querying a database, executing a blockchain transaction, running an AI model, sending a payment. Actions are versioned, namespace-scoped, and independently deployable. W3.io ships with native actions for Ethereum, Solana, and Bitcoin blockchain operations. Partners contribute actions for their specific capabilities.

Workflows are sequences of actions triggered by events. A workflow definition specifies a trigger (a cron schedule, an RPC call, a blockchain event, or an oracle feed), a series of steps that each invoke an action, and the data flow between steps. Workflows are compiled from YAML into a canonical form, deployed to the validator network, and executed with Byzantine fault-tolerant consensus on each step's result.

Solutions are workflows integrated into end-user applications. A treasury management application, a compliance pipeline, a cross-chain bridge: each is a solution built from workflows that compose

ecosystem actions. Solutions inherit the verifiability of the underlying protocol without requiring the end user to understand the execution mechanics.

This layered model means that ecosystem partners contribute capabilities once, developers compose them into workflows without custom integration code, and businesses deploy solutions that are verifiable by construction.

2.3 What This Paper Covers

The remainder of this paper describes the W3.io protocol in detail:

- **Architecture** (Section 2): system overview, data flow, and the relationship between protocol components
- **Workflows** (Section 3): the declarative syntax, trigger types, step kinds, and expression evaluation
- **Execution** (Section 4): how steps are isolated and executed on validators, with timeout enforcement and signal escalation
- **Consensus** (Section 5): the BOSCO BFT protocol, committee selection, and the four contexts in which consensus is used
- **Settlement** (Section 6): the cryptographic commitment structure that makes workflow execution independently verifiable
- **Cryptography** (Section 7): the specific algorithms used and their provenance in production-grade, audited implementations
- **Namespaces** (Section 8): the hierarchical multi-tenancy model for organizing workflows, billing, and access control
- **Validators** (Section 9): how operators join the network, declare capabilities, and maintain liveness
- **Ecosystem** (Section 10): the B2B2C partner model and the action registry
- **Token** (Sections 11-12): the utility of the W3 token within the protocol and its economic structure
- **Governance** (Section 13): how the protocol evolves and the decentralization roadmap
- **Security** (Section 14): the threat model, BFT assumptions, and known limitations

W3.io's protocol is implemented in Rust, open-source, and deployed on testnet. This paper describes the system as designed and implemented, not as aspirational. Where features are planned but not yet deployed, this is stated explicitly.

3 Architecture

3.1 Ingredients, Recipes, and Solutions

W3.io's ecosystem is organized around three tiers of composability.

Ingredients are capabilities provided by individual ecosystem partners. Each partner exposes their capabilities as modular, reusable components within the W3.io ecosystem. Space and Time provides data queries. Circle and Stripe handle payments. Hyperbolic offers AI inference. Chainalysis handles compliance screening. Pyth delivers price feeds. An ingredient is a self-contained unit of functionality that can be invoked as part of a larger process.

Recipes combine two or more ingredients into adaptive logic that addresses a specific business need. A recipe defines the trigger conditions, the sequence of operations, the data flow between steps, and the failure handling behavior. Recipes are W3.io's unit of deployment. A developer writes a recipe, deploys it to the validator network, and it executes automatically when triggered.

Solutions are recipes integrated into end-user applications. A payments product, a compliance pipeline, a yield vault: each is a solution that composes ingredients through recipes to deliver a complete user experience. Solutions inherit the verifiability of the underlying protocol. Every step of every recipe execution is attested by consensus and provable against an on-chain commitment.

This model enables a network effect. Each new ingredient makes every existing recipe potentially more capable, and each new recipe demonstrates a pattern that other builders can adapt.

3.2 Actions and Workflows

At the protocol level, ingredients become **actions** and recipes become **workflows**.

An action has a namespace-scoped identifier, a version, and defined inputs and outputs. W3.io ships with native actions for three blockchain families:

- **Ethereum:** balance queries, token transfers and approvals, contract deployment and calls, event monitoring, NFT operations, ENS name resolution
- **Solana:** balance queries, token accounts, program invocation, transfers, transaction monitoring
- **Bitcoin:** balance queries, UTXO management, fee estimation, transaction construction and broadcast

Ecosystem partners extend the action set by publishing their own actions. These are packaged as container images with a standard entry point, versioned independently, and invoked via the `uses:` step kind in workflow YAML. Current ecosystem actions include data queries (Space and Time), price feeds (Pyth), compliance screening (Chainalysis), payments (Circle, Stripe), AI inference

(Hyperbolic), caching (Redis), email delivery, and token transfers.

Workflows are defined in YAML using a syntax compatible with GitHub Actions:

```
name: Cross-chain settlement
on:
  schedule:
    - cron: '*/5 * * * *'
jobs:
  settle:
    steps:
      - name: Query pending transactions
        uses: w3-io/w3-sxt-action@v1
        with:
          command: query
          sql: >
            SELECT * FROM transactions
            WHERE status = 'pending'

      - name: Screen for compliance
        uses: w3-io/w3-chainalysis-action@v1
        with:
          address: ${ steps.query.outputs.sender }

      - name: Execute settlement
        ethereum:
          action: call-contract
          network: ethereum
          params:
            to: "0x..."
            function: "settle(address,uint256)"
            args:
              - ${ steps.query.outputs.sender }
              - ${ steps.query.outputs.amount }
```

The GitHub Actions compatibility is deliberate. Developers already familiar with CI/CD workflows can write W3.io workflows without learning a new language. The YAML is compiled into a canonical form, hashed for on-chain provenance, and deployed to the validator network.

3.3 The Three-Layer Data Architecture

W3.io separates data responsibilities across three layers, each optimized for its role.

Layer 1: L1 Settlement. The on-chain layer stores commitments, not data. The footprint is minimal: one cumulative root per epoch, plus contract state for the validator set and workflow registry. This keeps gas costs low while providing the anchor for cryptographic verification.

Layer 2: Protocol Nodes. Validators store everything. Full workflow execution traces, receipt storage, epoch accumulation, P2P consensus messages, and a write-ahead log for crash recovery. This layer is the evidence behind the Layer 1 commitments and serves as the data availability layer for proof generation.

Layer 3: Index and Query. An indexer watches Layer 1 events and reconstructs queryable views: workflow runs by namespace, by epoch, by action type. The CLI provides developer access to settlement status and proof export. The API layer supports MCP (Model Context Protocol) for AI agent integration.

3.4 The Spine and Ribs

W3.io's settlement architecture is best understood as a spine with ribs.

The Spine: L1 Epoch Chain

Epoch 0	Epoch 1	Epoch 2
root: 0xaaa	root: 0xbbb	root: 0xccc
prev: 0x000	prev: 0xaaa	prev: 0xbbb

The Ribs: Per-Run Hashchains

```

Workflow A -- block 1 -- block 2 -- receipt
Workflow B -- block 1 -- receipt
Workflow C -- block 1 -- block 2 -- block 3 -- receipt

```

The **spine** is the L1 epoch chain. A sequential, immutable record of cumulative sparse merkle tree roots on-chain. Each epoch's root covers all workflow receipts across all namespaces ever settled, not just the current epoch. The chain of roots captures the full evolution of world state.

The **ribs** are individual workflow execution traces. Per-run hashchains of consensus blocks that branch off the spine. Each step in a workflow produces a block containing the step's inputs, outputs, executor identity, and the previous block's hash. When the workflow completes, the final block hash and metadata are compressed into a receipt. That receipt is inserted into the namespace's sparse

merkle tree, which feeds into the global tree, which produces the epoch root committed to the spine.

Any workflow execution is provable against any epoch root that includes it. A verifier needs only the on-chain root, a receipt, and two merkle proofs (receipt-in-namespace, namespace-in-global) to confirm that a specific workflow executed with a specific result at a specific time. No re-execution required. No trust in the validator that ran it.

4 Workflows

4.1 Declarative Syntax

W3.io workflows are defined in YAML using the same syntax as GitHub Actions. This is not a loose compatibility. The W3.io compiler parses the same grammar, supports the same expression language, and handles the same structural elements: jobs, steps, conditionals, environment variables, secrets, outputs, and step references.

A developer who has written a GitHub Actions workflow can write a W3.io workflow without learning anything new. The difference is what happens after deployment. A GitHub Actions workflow runs on a centralized runner. A W3.io workflow runs on a distributed validator network with BFT consensus on every step.

The following example shows a non-trivial workflow that monitors an Ethereum wallet for large inbound transfers, screens the sender for sanctions compliance, logs the event to a database for audit, and sends a notification. It uses four ecosystem partners (Chainalysis for compliance, Space and Time for data, a Redis cache for deduplication, and an email action for alerting) alongside native Ethereum actions.

```
name: Large transfer compliance monitor
on:
  schedule:
    - cron: '*/5 * * * *'
jobs:
  monitor:
    steps:
      - name: Check recent transfers
        id: transfers
        uses: w3-io/w3-sxt-action@v1
        with:
          command: query
```



```

sql: >
    SELECT tx_hash, from_address, value
    FROM ethereum.erc20_transfers
    WHERE to_address = '${{ secrets.WATCHED_WALLET }}'
    AND value > 10000000000
    AND block_timestamp > NOW() - INTERVAL '5 MINUTES'

- name: Deduplicate against cache
  id: dedup
  if: steps.transfers.outputs.row_count > 0
  uses: w3-io/w3-redis-action@v1
  with:
    command: sismember
    url: '${{ secrets.REDIS_URL }}'
    key: processed-transfers
    value: '${{ steps.transfers.outputs.rows[0].tx_hash }}'

- name: Screen sender address
  id: compliance
  if: steps.dedup.outputs.result == '0'
  uses: w3-io/w3-chainanalysis-action@v1
  with:
    address: '${{ steps.transfers.outputs.rows[0].from_address }}'

- name: Log to audit trail
  if: steps.compliance.outputs.risk != 'sanctions'
  uses: w3-io/w3-sxt-action@v1
  with:
    command: execute
    sql: >
      INSERT INTO compliance_log
      (tx_hash, sender, amount, risk_level, checked_at)
      VALUES ('${{ steps.transfers.outputs.rows[0].tx_hash }}',
        '${{ steps.transfers.outputs.rows[0].from_address }}',
        '${{ steps.transfers.outputs.rows[0].value }}',
        '${{ steps.compliance.outputs.risk }}',

```

```

NOW())

- name: Mark as processed
  uses: w3-io/w3-redis-action@v1
  with:
    command: sadd
    url: ${ secrets.REDIS_URL }
    key: processed-transfers
    value: ${ steps.transfers.outputs.rows[0].tx_hash }

- name: Alert on high risk
  if: steps.compliance.outputs.risk == 'high'
  uses: w3-io/w3-email-action@v1
  with:
    to: compliance@example.com
    subject: >
      High risk transfer detected:
      ${ steps.transfers.outputs.rows[0].tx_hash }

```

Every step in this workflow executes on a different validator, selected by consensus. The compliance check result, the database write, and the alert are all attested by the committee and included in the settlement receipt. An auditor can later prove that the compliance check happened, what it returned, and when, by verifying the receipt against the on-chain epoch root.

A workflow definition includes:

- **Name:** human-readable identifier for the workflow
- **Trigger (on:):** the event that initiates execution
- **Jobs:** one or more named job blocks, each containing a sequence of steps
- **Steps:** ordered operations within a job, each invoking an action or running a command
- **Environment (env:):** key-value pairs scoped to the workflow, job, or individual step
- **Secrets:** sensitive values fetched at runtime from a secrets provider, never stored in the workflow definition

4.2 Triggers

Every workflow begins with a trigger. W3.io supports four trigger types.

Cron schedule (schedule). Time-based triggers that fire on a cron expression. A workflow triggered every five minutes, every hour, or once a day. The cron expression follows standard

POSIX syntax. Useful for batch processing, periodic data aggregation, and scheduled reporting.

```
on:
  schedule:
    - cron: '*/5 * * * *'
```

RPC dispatch (`workflow_dispatch`). On-demand triggers fired by an API call. A user, an application, or an AI agent sends a request to the W3.io RPC endpoint with optional input parameters. The workflow runs once with the provided inputs. Useful for user-initiated actions like deploying a contract, processing a payment, or running an analysis.

```
on:
  workflow_dispatch:
    inputs:
      token_address:
        description: 'Token to analyze'
        required: true
      chain:
        description: 'Blockchain network'
        default: 'ethereum'
```

Chain events. Blockchain event triggers that fire when a specific on-chain event occurs. An ERC-20 transfer, a contract deployment, a governance vote. The validator network monitors the specified chain for matching events and initiates the workflow when one is detected. Currently supported for Ethereum and Solana.

Oracle feeds. External data triggers based on oracle price updates or threshold crossings. When a Pyth price feed crosses a defined boundary, the workflow fires. Useful for automated trading, liquidation monitoring, and alerts.

4.3 Step Kinds

Each step in a workflow invokes one of five step kinds.

Run (`run:`). Executes a shell command inside an isolated container. The command runs in a Docker container with a defined image, environment variables, and mounted volumes. Output is captured from stdout and parsed as step outputs. This is the most flexible step kind, suitable for custom logic, data transformation, and any operation not covered by a specialized action.

```
- name: Process data
  run: |
    echo "Processing ${ steps.query.outputs.count } records"
```

```
python3 transform.py --input data.json
```

Uses (`uses:`). Invokes a published action by reference. The action is identified by its namespace, name, and version. Inputs are passed via the `with:` block. Outputs are available to subsequent steps via `{{ steps.<id>.outputs.<key> }}`. This is how ecosystem partner capabilities are accessed.

```
- name: Query blockchain data
  id: query
  uses: w3-io/w3-sxt-action@v1
  with:
    command: query
    sql: SELECT * FROM ethereum.transactions LIMIT 10
```

Ethereum (`ethereum:`). Native blockchain operations on Ethereum and EVM-compatible chains. Supports balance queries, token transfers and approvals, contract deployment and calls, event queries, NFT operations, and ENS name resolution. The `network` parameter selects the target chain.

```
- name: Check balance
  ethereum:
    action: get-balance
    network: ethereum
    params:
      address: "0x742d35Cc6634C0532925a3b844Bc9e7595f2bD18"
```

Solana (`solana:`). Native blockchain operations on Solana. Supports balance queries, token account lookups, program invocation, SOL and SPL token transfers, and transaction monitoring.

Bitcoin (`bitcoin:`). Native blockchain operations on Bitcoin. Supports balance queries, UTXO management, fee estimation, and transaction construction.

4.4 Expressions and Data Flow

W3.io's expression engine evaluates dynamic values at runtime using the `{{ }}` syntax. Expressions can reference:

- **Step outputs:** `{{ steps.query.outputs.result }}` accesses a named output from a previous step
- **Inputs:** `{{ inputs.token_address }}` accesses a trigger input parameter
- **Environment:** `{{ env.API_KEY }}` accesses an environment variable

- **Secrets:** `${{ secrets.PRIVATE_KEY }}` accesses a secret value (redacted in logs)
- **Job context:** `${{ job.status }}` reflects the current job state

The expression language supports functions for string manipulation (`contains`, `startsWith`, `endsWith`, `format`), data conversion (`toJSON`, `fromJSON`), and status checks (`success()`, `failure()`, `always()`).

Data flows between steps through outputs. Each step can produce named outputs that subsequent steps reference by step ID. This creates a typed data pipeline where each step receives exactly the data it needs from previous steps, without shared mutable state.

4.5 Conditionals and Failure Handling

Steps can be conditionally executed using the `if:` field.

```
- name: Alert on failure
  if: failure()
  uses: w3-io/w3-email-action@v1
  with:
    to: ops@example.com
    subject: Workflow failed
```

The `continue-on-error` field controls whether a step failure stops the workflow or allows subsequent steps to execute. Combined with status check functions (`success()`, `failure()`, `always()`), this enables error handling patterns like cleanup operations, fallback logic, and notification on failure.

Each step has a configurable timeout via `timeout-minutes`. If a step exceeds its timeout, the execution backend terminates it and the step is marked as failed. The default timeout is 10 minutes. The effective timeout for any step is the minimum of the step's configured timeout and the remaining time in the job's overall timeout.

4.6 Compilation and Deployment

Workflow YAML is not interpreted directly by validators. It goes through a compilation pipeline.

Parse: the YAML is parsed into an abstract syntax tree. Structural errors (invalid keys, missing required fields, malformed expressions) are caught here.

Validate: semantic validation checks type consistency, expression references, step ID uniqueness, and action compatibility. Invalid expression references (referencing a step that doesn't exist, or an output that an action doesn't produce) are caught before deployment.

Compile: the validated AST is compiled into a canonical form. Comments, formatting, and whitespace are stripped. The result is a deterministic representation of the workflow’s semantic content.

Hash: the compiled form is hashed. This `definition_hash` is the workflow’s identity for settlement purposes. Two workflows with identical semantics but different formatting produce the same hash. The hash is stored on-chain in the WorkflowRegistry contract, creating an immutable link between the deployed workflow and its on-chain provenance.

Deploy: the compiled workflow is distributed to the validator network. Validators store the definition and begin monitoring for the specified trigger events.